1) AVERAGE

- Returns the average value.

Eg: SELECT AVG(<column_name>)
FROM fake_apps;

—

5) ROUND

It takes 2 parameters.
One parameter is the column name, other is an integer which specifies the number of decimal places.

```
SELECT ROUND(price, 0)
FROM fake_apps;
```
→ round to 0 decimal places.

—

6) GROUP BY

→ It arranges identical data into groups.

Eg:
```
SELECT category, SUM(downloads)
FROM fake_apps
GROUP BY category;
```
Returns the total number of downloads for each category.

—

7) HAVING

- To filter groups.

—

LEARNING SQL DAY 4

⇛ Multiple tables:

Splitting the data into multiple tables.
So that information is not repeated in same table.

Tables: orders, subscriptions, customers

- Combining tables manually to understand info.
  ↳ Its called **joining two tables.**

- With SQL,

JOIN

↳ TO combine two tables:

```
SELECT *      → It selects all columns from the combined table
FROM orders   → Orders is the first table that we want to look into.
JOIN customers → we want to combine orders with customers.
  ON orders.customer_id = customers.customer_id; → we want to match
                                                   order's customer_id
                                                   with customer's customer_id.
```

Important NOTE : JOIN is also called INNER JOIN .
IF table1.column1 ≠ table2.column1
THEN, in the combined table, it skips that row.

To keep the unmatched rows, **LEFT JOIN**
LEFT JOIN keeps the rows of the first table despite it not matching.

SYNTAX :

```
SELECT *
FROM orders
LEFT JOIN customers
  ON table1.c2 = table2.c2;
```

PRIMARY KEY:

A primary key uniquely identifies each record in a table.

Requirements of a primary key:
- → Cannot be NULL
- → Unique value.
- → A table cannot have >1 primary key.

FOREIGN KEY:

→ When a primary key for one table appears in a different table, it is a **foreign key**.

Most JOINS joins primary keys with foreign key.

CROSS JOIN is used to join all rows of one table to all rows of another table.
↳ NO ON clause

Eg :

```
SELECT shirts.shirt_color,
  pants.pants_color
FROM shirts
CROSS JOIN pants;
```

UNION is used to stack one dataset on top of another.

Eg: table_1 :

pokemon	type
Bulbasaur	Gran
Charmander	Fire

table 2 :

pokemon	type
Snorlax	Normal

```
SELECT *
FROM table1
UNION
SELECT *
FROM table2
```

Pokemon	type
Bulbasaur	Gran
Charmander	Fire
Snorlax	Normal

Rules:

- Tables must have the same number of columns
- Datatypes should be in same order.